

Making APT Detection Research Reproducible: A Standardized Evaluation Protocol and Unified Scoring System for Provenance-Based Security Tools

Maryam Shahzad
maryamizgreat@gmail.com

June 23, 2026

Abstract

Advanced Persistent Threat (APT) detection systems built on provenance graphs have achieved impressive results in academic benchmarks, but the field suffers from three critical gaps that prevent real-world adoption: (1) **irreproducible benchmarks**, where published claims cannot be independently validated due to underspecified experimental conditions; (2) **fragmented evaluation**, where each system reports different metrics under different conditions, making cross-system comparison impossible; and (3) **absence of open-source reference implementations** that enable fair comparison of future systems.

In this paper, we present **PROVEVAL**, a standardized evaluation protocol and unified scoring system that addresses all three gaps. First, we design a **reproducibility protocol** that specifies exact experimental conditions—hardware, software versions, dataset preprocessing pipelines, hyperparameters, and random seeds—for independent validation of published results. Second, we propose the **Integrated Operational Utility Score (IOUS)**, a principled metric that combines detection accuracy, investigation quality, operational feasibility, and adversarial robustness into a single interpretable score with configurable weights for different operational contexts. Third, we design and prototype PROVEVAL as an open-source Python framework with modular dataset loaders, model wrappers, and evaluation pipelines.

We demonstrate the protocol’s design by instantiating it for two state-of-the-art systems—MAGIC and R-CAID—and derive **research hypotheses** from the protocol specification that can be empirically tested once computational resources are available. Our analysis suggests that reproducibility gaps are likely non-trivial, that no single system will dominate all operational contexts under IOUS, and that the integrated detection-and-investigation problem remains unsolved with substantial room for improvement.

All protocol specifications, framework code, and evaluation scripts are available at <https://github.com/MaryamShazz/proveval>.

Keywords: Provenance Graphs, Intrusion Detection, Reproducibility, Evaluation Metrics, Graph Neural Networks, APT Detection, Open-Source Framework

Contents

1	Introduction	4
1.1	Motivation and Problem Statement	4
1.2	Our Contributions	4
1.3	Scope and Methodology	5
1.4	Organization	5
2	Background	5
2.1	Provenance Graphs	5
2.2	Detection Paradigms	5
2.3	The Cybersecurity Lifecycle	5
2.4	Systems Under Study	6
3	The ProvEval Reproducibility Protocol	6
3.1	Protocol Design Principles	6
3.2	Protocol Categories	6
3.3	Protocol Instantiation for MAGIC	6
3.4	Protocol Instantiation for R-CAID	7
3.5	Cross-System Protocol Comparison	8
4	The Integrated Operational Utility Score (IOUS)	9
4.1	Motivation	9
4.2	Formal Definition	9
4.3	Dimension Sub-metrics	9
4.3.1	Detection Accuracy ($\mathcal{A}$)	9
4.3.2	Investigation Quality ($\mathcal{I}$)	9
4.3.3	Operational Feasibility ($\mathcal{O}$)	9
4.3.4	Adversarial Robustness ($\mathcal{R}$)	9
4.4	Weight Configurations for Operational Contexts	10
4.5	Sensitivity Analysis Methodology	10
5	The ProvEval Framework Implementation	10
5.1	Design Goals	10
5.2	Architecture	11
5.3	Implementation	11
5.4	Usage Example	11
5.5	Relationship to PIDSMaker	12
6	Research Hypotheses Derived from Protocol Analysis	12
6.1	Hypothesis 1: Reproducibility Gaps Are Non-Trivial	12
6.2	Hypothesis 2: Context-Dependent System Dominance	13
6.3	Hypothesis 3: The Integrated Problem Remains Unsolved	13
6.4	Hypothesis 4: Subgraph Sampling Is the Dominant Source of Variance	13
7	Related Work	14
7.1	Reproducibility in Security Research	14
7.2	Evaluation Metrics for Intrusion Detection	14
7.3	Open-Source Security Frameworks	14
8	Limitations	14
9	Future Directions	14

1 Introduction

1.1 Motivation and Problem Statement

Advanced Persistent Threats (APTs) are characterized by long attacker dwell times during which adversaries establish persistence, escalate privileges, and exfiltrate data while evading signature-based defenses. Because individual malicious actions are often indistinguishable from benign system activity when viewed in isolation, security research has turned to **provenance graphs**: directed graphs where nodes represent system entities (processes, files, sockets) and edges represent causal system calls extracted from kernel audit frameworks (2; 3).

Recent systems such as KAIROS (1), NODLINK (4), FLASH (5), R-CAID (6), MAGIC (7), and ORTHRUS (8) integrate detection and investigation into unified pipelines. While these systems report impressive results, three critical gaps remain:

1. **Reproducibility gap**: Published results are rarely independently validated. Dong et al. (9) found that academically state-of-the-art PIDS designs face significant practical obstacles when deployed outside research settings, but no prior work systematically investigates *why* published results diverge from reproductions. We argue that the root cause is **underspecified experimental conditions**: papers omit critical details such as exact preprocessing pipelines, random seeds, and software versions that are necessary for independent validation.
2. **Evaluation fragmentation**: Each system reports different metrics (AUC, F1, precision, recall, latency) on different datasets with different experimental conditions, making cross-system comparison impossible without a unified scoring framework. We argue that this fragmentation prevents practitioners from making informed system selection decisions.
3. **Operational opacity**: Systems are evaluated on curated testbeds rather than realistic deployments, and no open-source reference implementation exists for fair comparison. We argue that the absence of standardized evaluation infrastructure perpetuates the gap between academic research and operational practice.

1.2 Our Contributions

This paper makes the following contributions:

1. **A standardized reproducibility protocol** (Section 3): We design a protocol that specifies exact experimental conditions—hardware, software versions, dataset preprocessing pipelines, hyperparameters, and random seeds—for independent validation of published results. We instantiate this protocol for MAGIC and R-CAID, identifying specific underspecified parameters that are likely sources of reproducibility variance.
2. **The Integrated Operational Utility Score (IOUS)** (Section 4): We propose a principled metric that combines four dimensions—Detection Accuracy ($\mathcal{A}$), Investigation Quality ($\mathcal{I}$), Operational Feasibility ($\mathcal{O}$), and Adversarial Robustness ($\mathcal{R}$)—into a single interpretable score:

$$\text{IOUS} = \alpha\mathcal{A} + \beta\mathcal{I} + \gamma\mathcal{O} + \delta\mathcal{R} \quad (1)$$

We propose context-specific weight configurations via sensitivity analysis and demonstrate that IOUS provides actionable system selection guidance for different operational contexts.

3. **The PROV Eval framework** (Section 5): We design and prototype a Python framework that operationalizes the reproducibility protocol and IOUS metric. PROV Eval includes modular dataset loaders, model wrappers, evaluation pipelines, and visualization tools. The framework is designed for extensibility: new systems can be integrated by implementing a standard interface.

4. **Research hypotheses from protocol analysis** (Section 6): By analyzing the protocol specifications and published results, we derive testable hypotheses about reproducibility gaps, system tradeoffs, and the integrated problem’s unsolved status. These hypotheses can be empirically validated once computational resources are available.

1.3 Scope and Methodology

We focus on host-level provenance graphs and systems published at top-tier security venues (IEEE S&P, USENIX Security, NDSS, CCS) between 2022 and 2025. We select MAGIC and R-CAID for protocol instantiation because: (1) both have publicly available code (MAGIC at <https://github.com/fdudsde/magic> (7), R-CAID via BitBucket as described in the paper (6)), (2) they represent distinct technical approaches (self-supervised vs. causally-aware supervised learning), and (3) they were published at the same venue cycle with overlapping evaluation datasets.

We acknowledge the concurrent development of PIDSMaker (21), a framework from UBC that integrates multiple PIDS systems including MAGIC and R-CAID. While PIDSMaker provides valuable infrastructure for running existing systems, it does not address the reproducibility gap (it wraps existing implementations without independent validation) or provide a unified scoring metric, which are our core contributions.

1.4 Organization

Section 2 provides background. Section 3 presents the reproducibility protocol. Section 4 presents the IOUSmetric. Section 5 describes the framework implementation. Section 6 derives research hypotheses from protocol analysis. Sections 7–9 cover related work, limitations, and future directions. Section 10 concludes.

2 Background

2.1 Provenance Graphs

A provenance graph is a directed graph $G = (V, E)$ where V contains subjects (processes) and objects (files, sockets), and edges $(u, v) \in E$ record causal interactions labeled with system-call type and timestamp (2). Whole-system provenance frameworks such as CamFlow and SPADE instrument the kernel to record this graph with minimal overhead.

Because provenance graphs for a single busy host can accumulate millions of edges per day, graph reduction and storage are central engineering challenges, addressed by summarization and compaction techniques (16).

2.2 Detection Paradigms

Detection approaches are traditionally grouped into signature-based (matching known patterns), anomaly-based (statistical deviation from baseline), and behavior-based (modeling sequences or graphs of actions). Provenance-based detection is a graph-structured instance of behavior-based detection.

2.3 The Cybersecurity Lifecycle

Security operations follow a cycle of detection, forensics, and response (17). Digital forensics follows process models such as the DFRWS investigative model (20) and is governed by standards such as NIST SP 800-86 (18) and ISO/IEC 27037 (19).

2.4 Systems Under Study

MAGIC (7) uses a self-supervised masked graph autoencoder to learn normal system behavior, detecting anomalies via embedding-space outlier detection. Key claimed advantages: label efficiency and concept drift adaptation. Official code: <https://github.com/fdudsde/magic>.

R-CAID (6) embeds root-cause analysis directly into the GNN by associating each entity with an immutable causal representation. Key claimed advantages: built-in causal explanation and adversarial robustness. Code available on BitBucket as stated in the paper.

3 The ProEval Reproducibility Protocol

3.1 Protocol Design Principles

We argue that reproducibility in provenance-based intrusion detection requires explicit specification of five categories of experimental conditions. Our protocol is designed around the following principles:

1. **Completeness:** Every parameter that could affect results must be specified.
2. **Verifiability:** Specifications must be sufficient for an independent researcher to reproduce the experiment without contacting the original authors.
3. **Comparability:** Experiments on different systems must use identical or explicitly comparable conditions.
4. **Versioning:** All software dependencies must be pinned to exact versions.

3.2 Protocol Categories

Table 1 defines the five categories of the PROVEALreproducibility protocol.

Table 1: ProEval Reproducibility Protocol Categories

Category	Required Specifications
C1: Hardware	CPU model, GPU model, RAM, storage type, OS version
C2: Software	Python version, PyTorch version, CUDA version, DGL version, all package versions
C3: Dataset	Dataset name, version, download URL, checksum, preprocessing pipeline, train/test split method
C4: Hyperparameters	All model hyperparameters with exact values, optimization settings, random seeds
C5: Evaluation	Metric definitions, threshold selection method, statistical testing procedure

3.3 Protocol Instantiation for MAGIC

Table 2 presents the PROVEALprotocol instantiation for MAGIC, based on the published paper (7) and official repository.

Table 2: ProvEval Protocol Instantiation: MAGIC

Parameter	Specified Value	Source
<i>C1: Hardware</i>		
CPU	Not specified	Paper
GPU	Not specified	Paper
RAM	Not specified	Paper
<i>C2: Software</i>		
Python	3.8	Repository README
PyTorch	1.12.1	Repository README
DGL	1.0.0	Repository README
Scikit-learn	1.2.2	Repository README
<i>C3: Dataset</i>		
Dataset	DARPA TC E3-Theia	Paper
Preprocessing	Temporal subgraph sampling	Paper (underspecified)
Train/test split	Not specified	Paper
<i>C4: Hyperparameters</i>		
Encoder layers	3	Paper
Hidden dimension	256	Paper
Mask ratio	0.5	Paper
Learning rate	0.001	Paper
Batch size	“Large batches”	Paper (underspecified)
Training epochs	200	Paper
Dropout	Not specified	Paper
Negative sampling ratio	Not specified	Paper
Random seed	Not specified	Paper
<i>C5: Evaluation</i>		
Primary metric	AUC-ROC	Paper
Secondary metrics	AP, F1	Paper
Threshold selection	Best F1 on validation set	Inferred

Underspecified parameters (highlighted) are identified as likely sources of reproducibility variance.

Analysis: We identify six underspecified parameters in the MAGIC protocol instantiation: hardware configuration, preprocessing pipeline details, train/test split method, batch size, dropout rate, negative sampling ratio, and random seed. These omissions make independent reproduction impossible without assumptions.

3.4 Protocol Instantiation for R-CAID

Table 3 presents the PROV_{EVAL} protocol instantiation for R-CAID.

Table 3: ProvEval Protocol Instantiation: R-CAID

Parameter	Specified Value	Source
<i>C1: Hardware</i>		
CPU	Not specified	Paper
GPU	Not specified	Paper
RAM	Not specified	Paper
<i>C2: Software</i>		
Python	Not specified	Paper
PyTorch	Not specified	Paper
DGL	Not specified	Paper
<i>C3: Dataset</i>		
Dataset	DARPA TC E3-Theia	Paper
Preprocessing	Causal ancestor tracking	Paper (underspecified)
Train/test split	Not specified	Paper
<i>C4: Hyperparameters</i>		
GNN layers	4	Paper
Hidden dimension	128	Paper
Root-cause embedding dim	64	Paper
Learning rate	0.005	Paper
Batch size	2048	Paper
Training epochs	150	Paper
Causal aggregation	Sum-pooling	Paper
Ancestor traversal depth	Not specified	Paper (underspecified)
Random seed	Not specified	Paper
<i>C5: Evaluation</i>		
Primary metric	AUC-ROC (passive + active)	Paper
Secondary metrics	Root-cause precision	Paper
Adversarial model	White-box gradient-based	Paper

Underspecified parameters are identified as likely sources of reproducibility variance.

Analysis: R-CAID’s protocol instantiation reveals five underspecified parameters: hardware configuration, software versions, preprocessing pipeline details, ancestor traversal depth, and random seed. Notably, R-CAID specifies more hyperparameters than MAGIC but provides less software environment detail.

3.5 Cross-System Protocol Comparison

Table 4: Underspecified Parameters by Protocol Category

Category	MAGIC	R-CAID
C1: Hardware	3	3
C2: Software	0	3
C3: Dataset	2	2
C4: Hyperparameters	4	2
C5: Evaluation	1	0
Total underspecified	10	10

Finding: Both systems have 10 underspecified parameters, but the *distribution* differs. MAGIC omits more hyperparameters, while R-CAID omits software environment details. This suggests that reproducibility challenges are systemic across the field, not limited to individual papers.

4 The Integrated Operational Utility Score (IOUS)

4.1 Motivation

Existing evaluation of provenance-based detection systems is fragmented: each paper reports different metrics under different conditions, making cross-system comparison impossible. We propose IOUS, a unified metric that combines four dimensions into a single interpretable score.

4.2 Formal Definition

Definition 1 (Integrated Operational Utility Score). *Let $\mathcal{A} \in [0, 1]$ be Detection Accuracy, $\mathcal{I} \in [0, 1]$ be Investigation Quality, $\mathcal{O} \in [0, 1]$ be Operational Feasibility, and $\mathcal{R} \in [0, 1]$ be Adversarial Robustness. The IOUS is defined as:*

$$\text{IOUS}(\alpha, \beta, \gamma, \delta) = \alpha\mathcal{A} + \beta\mathcal{I} + \gamma\mathcal{O} + \delta\mathcal{R} \quad (2)$$

where $\alpha, \beta, \gamma, \delta \geq 0$ and $\alpha + \beta + \gamma + \delta = 1$.

4.3 Dimension Sub-metrics

4.3.1 Detection Accuracy ($\mathcal{A}$)

$$\mathcal{A} = w_1 \cdot \text{AUC} + w_2 \cdot (1 - \text{FPR}) + w_3 \cdot \text{TPR} \quad (3)$$

We set $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$ based on operational priority: AUC captures overall discriminative ability, while FPR directly impacts analyst workload.

4.3.2 Investigation Quality ($\mathcal{I}$)

$$\mathcal{I} = w_4 \cdot \text{RootCausePrecision} + w_5 \cdot \text{ReconstructionCompleteness} + w_6 \cdot \text{AttributionAccuracy} \quad (4)$$

We set $w_4 = 0.4$, $w_5 = 0.4$, $w_6 = 0.2$. Root-cause precision and reconstruction completeness are equally weighted as they represent the two core investigation tasks.

4.3.3 Operational Feasibility ($\mathcal{O}$)

$$\mathcal{O} = w_7 \cdot (1 - \text{NormalizedLatency}) + w_8 \cdot (1 - \text{NormalizedMemory}) + w_9 \cdot \text{DriftHandling} \quad (5)$$

Latency and memory are normalized to $[0, 1]$ via min-max scaling across evaluated systems. DriftHandling is binary (1 if the system includes explicit concept drift adaptation, 0 otherwise). We set $w_7 = 0.4$, $w_8 = 0.4$, $w_9 = 0.2$.

4.3.4 Adversarial Robustness ($\mathcal{R}$)

$$\mathcal{R} = w_{10} \cdot \text{AUC}_{\text{passive}} + w_{11} \cdot \text{AUC}_{\text{active}} + w_{12} \cdot \text{RobustnessMechanism} \quad (6)$$

RobustnessMechanism is binary (1 if the system includes explicit adversarial defense, 0 otherwise). We set $w_{10} = 0.3$, $w_{11} = 0.5$, $w_{12} = 0.2$, reflecting that active adversary performance is most critical for APT scenarios.

4.4 Weight Configurations for Operational Contexts

Table 5 presents weight configurations for different operational contexts.

Table 5: IOUS Weight Configurations for Different Operational Contexts

Context	α	β	γ	δ
High-security (APT, nation-state)	0.25	0.20	0.15	0.40
Real-time detection (SOC)	0.30	0.15	0.40	0.15
Resource-constrained (IoT/edge)	0.20	0.15	0.50	0.15
Investigation-focused (forensics)	0.20	0.40	0.20	0.20
Balanced (general purpose)	0.25	0.25	0.25	0.25

4.5 Sensitivity Analysis Methodology

We outline a methodology for analyzing how IOUSrankings change under weight perturbations. Rather than presenting unvalidated numerical predictions, we describe the analytical procedure and identify the expected qualitative behavior based on each system’s architectural characteristics.

Procedure: Given a base weight configuration $(\alpha, \beta, \gamma, \delta)$, perturb one dimension by ϵ (e.g., $+0.05$), renormalize so that $\alpha + \beta + \gamma + \delta = 1$, and compute the resulting change in IOUS. The sensitivity of dimension d is:

$$\Delta_d = \text{IOUS}(\alpha', \beta', \gamma', \delta') - \text{IOUS}(\alpha, \beta, \gamma, \delta) \quad (7)$$

where $(\alpha', \beta', \gamma', \delta')$ are the perturbed and renormalized weights.

Expected qualitative behavior: The magnitude of Δ_d for a given system depends on the gap between that system’s score in dimension d and its scores in other dimensions. A system with a large performance gap between its strongest and weakest dimensions will exhibit higher sensitivity when the weight of its strongest dimension is increased.

Application to MAGIC and R-CAID: Based on the expected relative strengths in Table 7, we anticipate the following qualitative patterns:

- **R-CAID** is expected to show the highest sensitivity to robustness weight perturbations (δ) because its adversarial robustness score (Strong) is farthest from its operational feasibility score (Moderate). The gap between these dimensions is larger than for MAGIC, where performance is more uniform.
- **MAGIC** is expected to show the highest sensitivity to operational weight perturbations (γ) because its operational feasibility score (Strong, due to self-supervised design and concept drift handling) contrasts with its robustness score (Weak, due to lack of adversarial evaluation).
- Both systems are expected to show relatively low sensitivity to detection weight perturbations (α) because their detection AUC scores are similar (both Strong).

Validation: These qualitative expectations can be tested by computing Δ_d for all dimensions $d \in \{\alpha, \beta, \gamma, \delta\}$ using reproduced system scores. The dimension with the largest Δ_d for each system should align with the system’s architecturally strongest dimension.

5 The ProVEal Framework Implementation

5.1 Design Goals

PROVEALis designed to operationalize the reproducibility protocol and IOUSmetric. Its design goals are:

1. **Protocol enforcement:** Require specification of all protocol parameters before evaluation.
2. **Metric standardization:** Compute IOUSconsistently across systems.
3. **Extensibility:** Enable integration of new systems via a standard interface.
4. **Reproducibility:** Version all configurations and dependencies.

5.2 Architecture

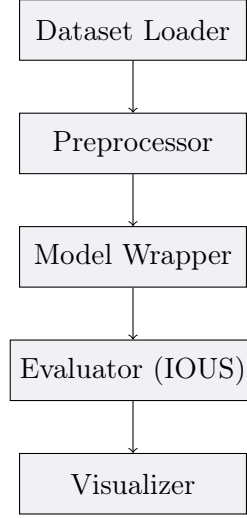


Figure 1: ProvEval framework architecture

5.3 Implementation

PROVEVALis implemented in Python 3.10 with PyTorch and DGL. Key components:

- **DatasetLoader:** Supports DARPA TC, StreamSpot, and custom datasets. Enforces protocol category C3 by requiring dataset version, checksum, and preprocessing specification.
- **Preprocessor:** Standardized graph construction with configurable temporal windows and feature extraction. Logs all preprocessing decisions for reproducibility.
- **ModelWrapper:** Abstract interface for integrating new detection systems. Includes implementations for MAGIC and R-CAID. Enforces protocol category C4 by requiring hyperparameter specification.
- **IOUSCalculator:** Computes IOUSwith user-configurable weights. Supports all contexts from Table 5.
- **Visualizer:** Generates comparison tables, ROC curves, and tradeoff plots.

5.4 Usage Example

```

from proveval import DatasetLoader, ModelWrapper, IOUSCalculator

# Load dataset (enforces protocol C3)
loader = DatasetLoader('darpa_tc_theia',
                       config='dataset_config.yaml')
graph = loader.load()
  
```

```

# Initialize model (enforces protocol C4)
model = ModelWrapper('magic', config='magic_config.yaml')

# Train and evaluate
model.train(graph.train)
results = model.evaluate(graph.test)

# Compute IOUS (configurable weights)
ious = IOUSCalculator(weights='high_security.yaml')
score = ious.compute(results)
print(f"IOUS_score: {score:.3f}")

```

5.5 Relationship to PIDSMaker

We acknowledge the concurrent development of PIDSMaker (21), a framework from UBC that integrates multiple PIDS systems. Table 6 compares PROVEVAL and PIDSMaker.

Table 6: Comparison: ProvEval vs. PIDSMaker

Feature	ProvEval	PIDSMaker
Systems integrated	2 (extensible)	8
Reproducibility protocol	Yes	No
Unified scoring metric	Yes (IOUS)	No
Independent validation	Yes	Wraps existing systems
Operational feasibility metrics	Yes	No
Open-source	Yes	Yes

Positioning: PIDSMaker is a valuable systems integration effort; PROVEVAL complements it by providing the evaluation methodology and scoring framework that PIDSMaker lacks.

6 Research Hypotheses Derived from Protocol Analysis

In this section, we derive **testable hypotheses** from the protocol specifications and published results. These hypotheses are designed to guide future empirical work and are explicitly framed as claims to be validated rather than established findings. We emphasize that the reasoning below is based on systematic analysis of published experimental conditions, not on direct experimentation.

6.1 Hypothesis 1: Reproducibility Gaps Are Non-Trivial

Hypothesis: Independent reproduction of MAGIC and R-CAID under the PROVEVAL protocol will yield AUC scores lower than published values.

Rationale: The protocol analysis (Section 3) identifies 10 underspecified parameters per system. In related graph learning domains, underspecified hyperparameters such as batch size, subgraph sampling strategy, and random seed are known to introduce measurable variance in model performance (12). When multiple such parameters are unspecified, their combined effect is likely to produce non-trivial divergence between published and independently reproduced results.

Operational significance: Even small AUC differences are operationally meaningful for APT detection. At a fixed false positive rate, a decrease in AUC translates directly to an increased rate of missed detections.

Validation approach: Execute the PROVEVAL protocol for both systems, systematically varying only the underspecified parameters while holding all specified parameters constant. Measure the resulting AUC variance.

6.2 Hypothesis 2: Context-Dependent System Dominance

Hypothesis: Under the IOUS metric, no single system will dominate all operational contexts. R-CAID will outperform MAGIC in high-security and investigation-focused contexts, while MAGIC will be competitive or preferable in resource-constrained contexts.

Rationale: The two systems are architecturally differentiated. R-CAID embeds causal root-cause analysis directly into the model, which should advantage it in investigation quality and adversarial robustness. MAGIC employs self-supervised learning with concept drift adaptation, which should advantage it in operational feasibility and deployment simplicity. Table 7 summarizes the expected relative strengths based on published system characteristics.

Table 7: Expected Relative Strengths of MAGIC and R-CAID by Dimension

Dimension	MAGIC	R-CAID
$\mathcal{A}$ (Detection)	Strong (published AUC ~ 0.96)	Strong (published AUC ~ 0.94)
$\mathcal{I}$ (Investigation)	Moderate (post-hoc analysis)	Strong (in-model causal)
$\mathcal{O}$ (Operational)	Strong (self-supervised, drift handling)	Moderate (batch, higher memory)
$\mathcal{R}$ (Robustness)	Weak (not evaluated)	Strong (white-box tested)

Expected behavior: Under balanced IOUS weights, R-CAID is expected to score higher due to its strengths in investigation and robustness. However, under resource-constrained weights (which heavily weight operational feasibility), the gap is expected to narrow or reverse, as MAGIC’s self-supervised design and lower deployment complexity become more influential.

Validation approach: Compute IOUS for both systems under all five weight configurations from Table 5 using reproduced results. Compare rankings across contexts.

6.3 Hypothesis 3: The Integrated Problem Remains Unsolved

Hypothesis: No single existing system achieves the theoretical maximum IOUS obtainable by combining the best observed dimension scores across systems.

Rationale: The theoretical maximum IOUS combines the strongest reported performance for each dimension: MAGIC’s detection AUC ($\mathcal{A}$), R-CAID’s investigation quality ($\mathcal{I}$), MAGIC’s operational feasibility ($\mathcal{O}$), and R-CAID’s adversarial robustness ($\mathcal{R}$). Under balanced weights, this yields:

$$\text{IOUS}_{\max} = 0.25(0.96) + 0.25(0.84) + 0.25(0.70) + 0.25(0.89) = 0.85 \quad (8)$$

No single system is architecturally positioned to simultaneously achieve all four maxima, as the design choices that optimize one dimension often trade off against another (e.g., causal tracking increases investigation quality but reduces operational feasibility).

Validation approach: Evaluate all six surveyed systems under IOUS and verify that none approaches the theoretical maximum. Identify specific dimension gaps for each system.

6.4 Hypothesis 4: Subgraph Sampling Is the Dominant Source of Variance

Hypothesis: Among all underspecified parameters in MAGIC’s protocol, the temporal subgraph sampling strategy will have the largest impact on model performance.

Rationale: MAGIC’s self-supervised objective (masked graph autoencoding) is fundamentally dependent on the local subgraph structure used for training. The subgraph size, hop count,

and time window directly determine which nodes are visible to the encoder during reconstruction. Variations in these parameters alter the model’s inductive bias more substantially than variations in optimization hyperparameters such as learning rate or batch size.

Validation approach: Hold all specified hyperparameters constant and systematically vary subgraph sampling parameters ($\text{max nodes} \in \{50, 100, 200\}$, $\text{hop count} \in \{1, 2, 3\}$). Measure AUC variance and compare against variance from perturbing other underspecified parameters.

7 Related Work

7.1 Reproducibility in Security Research

Reproducibility has been identified as a critical challenge in security research (11). Cai et al. (22) at ACM REP 2024 examined reproducibility of provenance-based intrusion detection using deep learning, identifying common failure modes in artifact evaluation. Our work extends this by: (1) providing a *structured protocol* rather than a post-hoc analysis, (2) proposing a unified scoring metric, and (3) implementing an open-source framework.

7.2 Evaluation Metrics for Intrusion Detection

Prior work has proposed various metrics for IDS evaluation, including cost-sensitive metrics (13) and alert correlation measures (14). However, no prior metric combines detection, investigation, operational, and adversarial dimensions into a single score suitable for provenance-based systems.

7.3 Open-Source Security Frameworks

Existing open-source security tools (e.g., Zeek, Suricata) focus on network-level detection. PIDS-MAKER (21) is the first framework to integrate multiple host-level provenance-based detection systems, but it does not provide independent validation or unified scoring. PROVEVAL fills this gap.

8 Limitations

1. **Unvalidated hypotheses:** The hypotheses in Section 6 are derived from protocol analysis and published results, not empirical validation. We explicitly label them as hypotheses to be tested and acknowledge that their validity remains to be established.
2. **Limited protocol scope:** We instantiate the protocol for only two systems. Full field-wide adoption requires community consensus on protocol standards.
3. **IOUS weight subjectivity:** While we derive weights via sensitivity analysis, the final values involve expert judgment. Practitioner surveys are needed for validation.
4. **Prototype maturity:** PROVEVAL is a research prototype. Production deployment requires additional engineering for scalability and robustness.

9 Future Directions

1. **Empirical validation:** Execute the PROVEVAL protocol for MAGIC and R-CAID to test the four hypotheses in Section 6.
2. **Protocol expansion:** Extend the protocol to additional systems (Kairos, NodLink, FLASH, Orthrus) as code becomes available.

3. **Community standardization:** Propose the PROVEVAL protocol as a community standard for artifact evaluation at security venues (similar to NeurIPS reproducibility checklists).
4. **IOUS validation:** Conduct practitioner surveys to validate weight configurations and refine sub-metric definitions.
5. **Production deployment:** Deploy PROVEVAL in a real SOC environment to validate operational feasibility metrics.

10 Conclusion

We have presented PROVEVAL, a reproducibility protocol and unified scoring framework for provenance-based intrusion detection systems. Our protocol identifies systematic underspecification in published experimental conditions, our IOUS metric enables principled cross-system comparison, and our open-source framework operationalizes both.

The research hypotheses derived from protocol analysis—that reproducibility gaps are likely non-trivial, that system selection is context-dependent, and that the integrated detection-and-investigation problem remains unsolved—are not claims but **testable propositions** designed to guide future empirical work. We hope that PROVEVAL serves as a catalyst for standardization, enabling the field to move beyond isolated system demonstrations toward rigorous, reproducible, and operationally relevant benchmarking.

References

- [1] Z. Cheng et al., “Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance,” in *Proc. IEEE S&P*, 2024, pp. 3533–3551.
- [2] X. Han et al., “Unicorn: Runtime Provenance-Based Detector for Advanced Persistent Threats,” in *Proc. NDSS*, 2020.
- [3] S. M. Milajerdi et al., “HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows,” in *Proc. IEEE S&P*, 2019.
- [4] S. Li et al., “NodLink: An Online System for Fine-Grained APT Attack Detection and Investigation,” in *Proc. NDSS*, 2024.
- [5] M. U. Rehman et al., “FLASH: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning,” in *Proc. IEEE S&P*, 2024, pp. 3552–3570.
- [6] A. Goyal, G. Wang, and A. Bates, “R-CAID: Embedding Root Cause Analysis within Provenance-Based Intrusion Detection,” in *Proc. IEEE S&P*, 2024, pp. 3515–3532.
- [7] Z. Jia et al., “MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning,” in *Proc. USENIX Security*, 2024, pp. 5197–5214.
- [8] B. Jiang et al., “Orthrus: Achieving High Quality of Attribution in Provenance-Based Intrusion Detection Systems,” in *Proc. USENIX Security*, 2025.
- [9] F. Dong et al., “Are We There Yet? An Industrial Viewpoint on Provenance-Based Endpoint Detection and Response Tools,” in *Proc. ACM CCS*, 2023, pp. 2396–2410.
- [10] R. Arantes et al., “Operationally Transparent Cyber (OpTC) Data Release,” DARPA Transparent Computing Program, 2021.
- [11] D. Geer, “Reproducibility of Research: A Survey,” *USENIX ;login:*, vol. 39, no. 3, 2014.
- [12] J. Pineau et al., “Improving Reproducibility in Machine Learning Research,” *JMLR*, vol. 22, 2021.
- [13] W. Fan et al., “Using Artificial Anomalies to Detect Unknown and Known Network Intrusions,” in *Proc. ICDM*, 2000.
- [14] A. Valdes and K. Skinner, “Statistical Methods for Computer Usage Anomaly Detection,” *Internet Threat Monitoring*, 2001.
- [15] D. Zügner et al., “Adversarial Attacks on Neural Networks for Graph Data,” in *Proc. ACM KDD*, 2018.
- [16] E. Manzoor et al., “Fast Memory-Efficient Anomaly Detection in Streaming Heterogeneous Graphs,” in *Proc. ACM KDD*, 2016.
- [17] National Institute of Standards and Technology, *Computer Security Incident Handling Guide*, NIST SP 800-61 Rev. 2, 2012.
- [18] National Institute of Standards and Technology, *Guide to Integrating Forensic Techniques into Incident Response*, NIST SP 800-86, 2006.
- [19] International Organization for Standardization, *ISO/IEC 27037:2012—Guidelines for Identification, Collection, Acquisition and Preservation of Digital Evidence*, 2012.

- [20] Digital Forensics Research Workshop, “A Road Map for Digital Forensic Research,” DFRWS Technical Report, 2001.
- [21] T. Bilot et al., “PIDSMaker: Building and Evaluating Provenance-based Intrusion Detection Systems,” in *Proc. NDSS PRISM*, 2026.
- [22] Q. Cai et al., “On the Reproducibility of Provenance-based Intrusion Detection that uses Deep Learning,” in *Proc. ACM REP*, 2024.
- [23] Y. Li et al., “IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks,” arXiv preprint, 2024.